GlowProof

Technical Architecture

Prepared for judges - DevNetwork [API + Cloud + AI] Hackathon 2026. By HJM Technologies. Public demo: glow-proof.vercel.app · Source: github.com/harrymakoni-netizen/glow-proof

---

1. System overview

GlowProof turns one selfie into measured skin scores via a real computer-vision API (Perfect Corp), then produces a skincare routine and finds real, currently-priced products for it - without ever naming a brand internally. The architecture is built around one constraint: the app sells nothing, so no component in the pipeline is allowed to have an opinion about which product to prefer.

Backend: FastAPI (Python). Frontend: no-build vanilla JS/HTML/CSS. Persistence: Xano. Deployment: Vercel (zero-config Python/ASGI). Four external APIs, each independently optional and independently degradable, so the app never has a single point of failure that produces a broken screen.

Pipeline

```
selfie --> Perfect Corp Skin Analysis --> scores + mask overlays
                                          |
                                          +--> Xano (persistence: one row per scan)
                                          v
                      Claude / Gemini (structured output)
                      names WHAT is needed: product type + active ingredient
                      -- never a brand --
                                          v
                      SerpApi Google Shopping
                      finds WHICH real product satisfies it, at today's price
```

Splitting “what” from “which” across two independent systems is the core architectural decision: the LLM is structurally prevented from inventing a product that doesn't exist or from favoring a brand it has memorized from training data, because it is only ever asked for a category and an ingredient. A second, separate live lookup resolves that into an actual purchasable item.

2. Component breakdown

File	Responsibility
`app/config.py`	Environment loading and mode detection. Every external integration (skin analysis, LLM, product search, persistence) resolves to a boolean “live” flag at import time; the app never guesses at request time whether a key is usable.
`app/perfectcorp.py`	Perfect Corp YouCam Skin Analysis client: presigned-upload, task creation, poll-to-completion, response parsing into a typed Analysis/Concern model.
`app/routine.py`	Turns measured scores into a structured routine via Claude or Gemini (Pydantic-validated output). Contains the claims-discipline system prompt and a deterministic canned fallback for offline demoing.

app/products.py	SerpApi Google Shopping lookup with request de-duplication, a ThreadPoolExecutor fan-out for parallel searches, and a two-tier disk+memory cache.
app/xano.py	Persistence client against a hand-written XanoScript API (not Xano's default per-table REST CRUD) - create / get / save_routine / list.
app/main.py	FastAPI routes. Analysis and routine generation are deliberately separate endpoints (see §4).
static/	No-build frontend: vanilla JS, hand-authored CSS, single HTML shell with view-based sections toggled by class.

3. API surface

Endpoint	Purpose
GET /api/health	Reports live/fixture status per integration - the mode banner.
POST /api/analyze	Accepts an image upload, runs Perfect Corp analysis (or fixture), persists the scan to Xano if configured, returns scores + session id.
GET /api/scan/{id}	Returns the raw analysis for a session - used to reopen a past scan from history without re-running Perfect Corp.
GET /api/routine/{id}	Generates (or returns the cached) routine for a session, then enriches each product step with a live SerpApi result.
GET /api/history	Lists recent scans from Xano for the “Recent scans” panel.

/api/analyze and /api/routine are intentionally separate calls rather than one combined endpoint: scores paint on screen the instant Perfect Corp returns, and the routine generates underneath. Stacking two slow, sequential upstream calls behind a single spinner was rejected as a demo-reliability risk.

4. Persistence architecture (Xano)

Sessions originally lived only in an in-process dictionary - gone on restart, invisible across serverless cold starts, no way to revisit a result. Xano replaces this with a single `scans` table (id, created_at, analysis JSON, routine JSON nullable) and a hand-written XanoScript API, chosen over Xano's default per-table REST CRUD because two behaviors needed exact control:

- `get` returns HTTP 200 with a null body for a missing id, not a 404 - the client checks the body, not the status code.
- `save_routine` merges a routine onto an existing row without requiring the caller to resend the full record.

Schema changes go through Xano's own safety flow, not a direct write: edit the pulled workspace locally (`xano-workspace/`, itself a separate git repo), `xano sandbox push` to stage, then a manual `xano sandbox review` → Review & Push in the browser promotes it to production. No CLI path exists to skip that manual promotion step - it is Xano's own guard against an agent (or a script) silently mutating a live workspace.

The app degrades gracefully when Xano is unconfigured or a call fails mid-request: it falls back to the in-memory dict, same failure-isolation pattern used for every other integration (see §6).

5. Deployment architecture (Vercel)

Deployed as a zero-config Python/ASGI Vercel Function: a root-level `main.py` re-exports the FastAPI `app` instance from `app/main.py`, which is the convention Vercel's current FastAPI detection expects (no `vercel.json` rewrites - an earlier attempt using a custom `api/index.py` + rewrite broke routing, because Vercel now routes using the rewritten destination path rather than the original request path).

One filesystem constraint required a code change: Vercel's deployment filesystem is read-only outside `/tmp`. The SerpApi disk cache in `products.py` now writes to `/tmp` when `VERCEL` is set, while still reading the bundled seed file on a cold start - a cache-write failure was never allowed to break a live product lookup, so the write is also wrapped defensively.

The production deployment forces fixture mode (`GLOWPROOF_FORCE_FIXTURE=1`) regardless of local development settings, since the URL is public: nothing should let an anonymous visitor spend real Perfect Corp analysis credit.

6. Failure isolation / mode detection

Every external dependency resolves to an explicit, independently-checkable live/fixture state at startup, surfaced verbatim in `/api/health`'s mode banner. No code path silently pretends a live call succeeded when it didn't:

- No Perfect Corp key, or `GLOWPROOF_FORCE_FIXTURE=1` → canned fixture analysis.
- No LLM key → a deterministic, non-personalized canned routine (still coherent, just not tailored - proof the flow demos honestly offline).
- No SerpApi key, or a request failure → cached results, or an empty result set - never a crash.
- No Xano config, or a call failure → in-memory session store for that process.

This matters beyond robustness: a hackathon demo that dies on a third-party hiccup on stage is a worse failure mode than one that visibly degrades to “sample data” and keeps going.

7. Engineering note: a bug only a real API call could surface

The team deliberately gated real Perfect Corp calls behind `GLOWPROOF_FORCE_FIXTURE` to conserve a limited unit budget during development, iterating against fixtures instead. One real capture, run deliberately once real integration work was otherwise complete, surfaced a defect fixtures could never have shown: Perfect Corp's response array mixes non-concern housekeeping entries (`all`, `skin_age`, `resize_image`) into the same list as genuine skin concerns, each scored 0.

The original parser treated every entry in that array as a concern. Because “priorities” sorts ascending by score, those three zero-score housekeeping entries were silently outranking every real measurement - the generated routine would have targeted concerns literally named “All” and “Resize Image” instead of the person's actual pores, wrinkles, texture, and acne scores. The fix filters the response array to the concern types actually requested before building the concern list.

This is cited here not as a curiosity but as the argument for the fixture-gating strategy itself: a small, deliberate real-integration budget spent late, once fixture development was otherwise complete, is what catches a class of bug that mocked responses structurally cannot.

8. Security and privacy posture

- The uploaded selfie is never persisted anywhere in the stack - not on disk, not in Xano, not in logs. Only Perfect Corp's numeric analysis output is retained.
- Xano's scan-persistence endpoints carry no authentication by design: the stored payload is analysis scores and a generated routine, no PII, so the attack surface of adding auth was judged not worth the added complexity under a one-week deadline.
- All API credentials are environment variables, never committed - verified by a pre-push secret scan before the repository was made public.
- The claims-discipline system prompt in routine.py hard-forbids diagnosis language, disease names, brand names, and unhedged efficacy claims; the disclaimer is permanent UI, not a dismissible footer.

9. Stack summary

Layer	Technology
Language / runtime	Python 3.12 (backend), vanilla JavaScript (frontend, no build step)
Web framework	FastAPI + Pydantic (structured LLM output validation)
Skin analysis	Perfect Corp YouCam Skin Analysis API (S2S v2.0)
Routine generation	Anthropic Claude (structured output) or Google Gemini - provider chosen at startup by whichever key is present
Product search	SerpApi — Google Shopping engine
Persistence	Xano (hand-written XanoScript API + table, version-controlled workspace)
Deployment	Vercel — zero-config Python/ASGI Function
Source control	Git + GitHub

Full source, commit history, and the XanoScript backend definitions are available at github.com/harrymakoni-netizen/glow-proof. VERIFIED.md in that repository documents every claim in this document against a dated, real API response.